Cyber Defense Club - Session Guide (Oct 24, 2026)

This is the written reference for the Oct 24 session. There's no recording of this one, so this guide covers each activity we did - the commands, the reasoning, and what went wrong on purpose and by accident. Use it to review, or to catch up if you missed the meeting.

1. VM display: switch from Spice to VirtIO GPU

When we built our VMs we picked Spice for the display. Iowa State suggests VirtIO GPU instead, and it has been noticeably more stable. You still view the VM with Virt Viewer either way.

To change it: open your VM's hardware settings and change Display from Spice to VirtIO GPU. If the VM is running, the new setting shows up in red - that means it won't take effect until you restart the VM. If your VM is shut down, the change applies as soon as you start it.

Once you are running in Spice, icons and other parts of the desktop may look very small. To fix this, right-click on the desktop settings and choose Display Options and increase the Scale setting. 200% works great for me. Don't choose fractional scaling. It can slow down your VM and cause other display issues.

One side note: Spice allocates only 16 MB of display memory; VirtIO GPU gets 256 MB. This isn't mainly a performance change, but sessions hold up better.

Do this during the week, whenever your VM is shut down anyway.

2. The ip command and what /24 means

`ip` is a command with many subcommands (`ip address`, `ip route`, and more). The one you'll use constantly is `ip address` - most people shorten it to `ip a`, but typing `ip addr` helps you remember what it stands for.

Two things to notice in the output:

- **127.0.0.1/8** shows up on every computer. That's the loopback address - the address a computer uses to talk to itself. Traffic to it never goes out to a router; it stays on the local host.
- **Our VM addresses** look like 144.76.90.X/24, where X is your VM number. The `/24` is the prefix length: the first 24 bits (the first three numbers, 144.76.90) identify the *network*, and the last number identifies the *host* on that network.

Know this cold. It will be a question until nobody misses it. (Claude wrote that sentence. What does it mean?)

3. What went wrong last week: /24 became /0

Last week the network config looked right but didn't behave right. Here's what happened: the address was entered in the GNOME network settings in a way that ended up saved as **/0** instead of /24, and the dialog accepted it without complaint.

Think about what /0 means. Zero bits identify the network - so the *entire* address is treated as the host part. The VM concludes that nothing else is on its network. Literally nothing except itself is "local."

So when this VM tries to reach another VM that is actually sitting right next to it on the same subnet, it doesn't ARP for it directly - it hands the packet to the gateway. The gateway can forward it, but the reply path breaks down because the two machines disagree about what the network is. The result is asymmetric: a ping test showed packets transmitted and **zero** received.

The lesson: the subnet mask has to match the actual network, or routing decisions go sideways in ways that are hard to spot. The address can be perfectly correct and the machine still can't talk to its neighbors.

4. The correct network settings

The fixed configuration for the demo VM:

- Address: `144.76.90.117` (your last number is your VM ID)
- Subnet mask: `255.255.255.0` (equivalent to /24)
- Gateway: `144.76.90.254`
- DNS: `199.100.16.100`

The GUI accepts either `24` or the dotted form `255.255.255.0`. However, entering `/24` (with the leading slash), causes the /0 problem. Use the full dotted mask - it's unambiguous. After applying, confirm with `ip a` that the /24 is really there, and ping another VM to prove traffic flows both ways.

5. wget and the missing CSS

We used `wget` to download Google's home page:

```
wget https://www.google.com
```

That saves the HTML to a local file (index.html, or index.html.1, .2, ... if you've done it before). You can open it in Firefox with a file URL:

```
file:///path/to/saved/file.html
```

It looks like Google, but plain - no logo, no styling. Why? Because `wget` fetched only the HTML. The page's appearance comes from separate CSS (cascading style sheet) files, which the HTML references with relative URLs. When your browser opens a local file, those relative references point into your file system, where the CSS doesn't exist. (The search box doesn't work for the same reason - you have a dead local copy, not the live site.)

You *can* embed styles directly in HTML, but doing a whole site that way is error-prone and tedious, so real sites put CSS in separate files - which is exactly why our one-file download looks naked.

6. What SSH is

SSH stands for **Secure Shell**. It lets you get a shell on another computer from your own - full command-line access over the network, encrypted. You use it because you can't always walk (or fly) to the machine you need to work on.

The SSH *client* comes preinstalled on Ubuntu. Try it: type `ssh` with nothing after it. If you get a usage/help screen, you have the client. The *server* side is a separate package - that's what we set up next.

7. Turning on the SSH server with systemctl

On the *server* VM it was installed but **disabled** - meaning it would not start at boot. On the *client/desktop* VM, the SSH server was not installed. If you want to run it on an Ubuntu desktop computer, you need to install it with `apt`:

```
sudo apt update
sudo apt install openssh-server
```

`systemctl` is how you control services on modern Linux. Check the SSH service:

```
systemctl status ssh.service
```

To start the SSH server right now, run this:

```
sudo systemctl start ssh.service    # start it right now
```

To set it to run automatically every time the computer starts, run this:

```
sudo systemctl enable ssh.service   # start automatically at every boot
```

Run `systemctl status ssh.service` again and you should see both: "enabled" (the boot setting) and "active (running)" (right now). Note the timestamps are in UTC, so don't panic when the time looks five hours off.

Tip from the session: these remote consoles sometimes lag or repeat keystrokes, so becoming root once (`sudo -i`) beats retyping your password for every command.

8. Installing OpenSSH server on a fresh VM

On a VM that doesn't have the server installed yet, there's one rule: **update the package catalog before you install anything.**

```
sudo apt update
sudo apt install openssh-server
```

`apt update` refreshes the package metadata. Skip it and installs can fail because dependency information has gone stale - there is *always* some package that's been updated since your VM was built. After installing, enable and start the service exactly as in section 7.

9. Your first connection and the host key prompt

From your own terminal, run this command. `<username>` is the name of your SSH user on the remote computer (not the name you used to log onto your current computer).

```
ssh yourusername@144.76.90.117
```

The first time you connect to any server, SSH shows you the server's key fingerprint (a SHA-256 hash) and asks if you want to continue. This is SSH asking: are you sure this is really the machine you meant to reach?

When you answer yes, SSH stores that fingerprint in a local file, `~/.ssh/known_hosts`. Every future connection to that address is checked against the stored fingerprint. If it ever *changes*, SSH throws a loud warning - because a changed key can mean you're talking to a different machine than before, possibly an impostor. You then have to decide: did I knowingly rebuild that server, or should I be worried?

In our case we were all connecting to a known server on our own network, so answering yes was safe.

10. Why port 22?

Linux keeps a list of well-known port assignments in `/etc/services`. You can find SSH's entry with grep:

```
$ grep '\b22\b' /etc/services
ssh    22/tcp    # SSH Remote Login Protocol
```

The line with 22 by itself shows `ssh ... 22/tcp` described as remote login. The file is a little dated, but it's a handy reference for protocols that have been around a while. (More on whether moving off port 22 helps you in section 14 - short answer: not really.)

11. Adding users

To let people SSH in, they need accounts on the server. In the GUI: Settings, then Users, unlock with your password, and Add User. Give each person a username and a password - Ubuntu enforces a minimum length and will push back on weak choices (like a password that contains the username).

Each new user gets a standard, unprivileged account. We'll look at what that account actually consists of in section 13.

12. Connecting with your username

The full connection syntax is:

```
ssh yourusername@144.76.90.117
```

Replace `yourusername` with your actual account name - the literal word "username" will not go well. You'll be prompted for your password, and then you have a shell on the server.

Two commands help when you're not sure which machine you're sitting on (easy to lose track of inside nested sessions): `hostname` tells you the machine's name, and `ip address` shows its address.

Fun fact: you can SSH to the machine you're already on (`ssh you@its-own-address`). That sounds pointless, but it's a quick way to test that the SSH service itself is working when someone reports they can't connect from outside.

13. Users, /etc/passwd, /etc/shadow, and permissions

Home directories. A new user gets a home directory at `/home/username`. Fresh out of the box it contains only default configuration files. And with default permissions, you cannot `cd` into *another* user's home directory - accounts are isolated from each other.

/etc/passwd. This file lists every account: username, UID, primary group, home directory, and default login shell (ours is `/bin/bash`). The second field is just an `x`. That's a placeholder - this file *used* to hold password hashes, but a real hash is a long string, and that lone `x` says "the hash lives somewhere else now." The file stayed world-readable because various programs need to look up account info.

/etc/shadow. This is where the hashes actually live. It's owned by root, group `shadow`, and ordinary users can't read it - you need root. Inside, real users have long hash strings. Accounts that show `!` or `*` where the hash would be are service accounts: they can't log in interactively at all, but root can still run processes under them. That's a deliberate security design - a web server account that nobody can log into is an account nobody can steal a password for.

/etc/group. This is where group memberships are defined. For a user to have SSH privileges enabled, they must be assigned into the `_ssh` group. Here are the members of the `_ssh` group on 144.76.90.117, straight from the `/etc/group` file:

```
$ grep '_ssh' /etc/group
_ssh:x:104:isaiah,naomi,kizek,chris,karl,hans,generic_user
```

Reading permissions. In `ls -l` output, after the first special character, the next nine characters come in three groups of three:

- first three: what the **owner** can do
- middle three: what the file's **group** can do
- last three: what **everyone else** can do

Each group uses `r` (read), `w` (write), `x` (execute). So `/etc/shadow` showing owner `rw`, group `r`, others nothing - with owner `root` and group `shadow` - means root can read and write it, members of the shadow group can read it, and everyone else gets denied.

14. A look ahead: sshd_config

SSH's configuration lives under `/etc`. There are two sides to keep straight: `ssh` (and ssh_config) is the *client*; `sshd` is the *server*. The D stands for **daemon** - the old Unix word for a service that runs on its own in the background. There's also a directory, `sshd_config.d`, where additional server config files can be dropped in (ours is empty so far).

The main `sshd_config` is about 131 lines, mostly commented-out examples. One worth discussing now: you *can* uncomment `Port 22` and move SSH to some high port like 50003. Every year somebody wants to, thinking it hides the service. It doesn't. Scanners don't look for port 22 - they look for anything that *behaves*

like SSH, and the service announces itself with a banner on connection. A nonstandard port buys you a little less noise in the logs, not security.

Next week we'll walk through the config for real: ListenAddress (and why there are two, plus whether we care about IPv6), host keys, LoginGraceTime, and AuthorizedKeysFile. The big one is **public-key authentication**: you keep a private key, you install the matching public key on the server, and when they match you log in without ever sending a password.

That leads to a question defense teams don't think about enough: how do you protect root? Even if nobody can log in as root directly, an attacker who compromises a regular account can escalate through a sudo password or an installed key. One strong extra control is a one-time password - the Google Authenticator style TOTP codes, which SSH can require through a config file we'll look at. An attacker with a stolen password still doesn't have the rotating 6-digit code. (House rule for competitions: if the green team doesn't have to touch it, it's fair game to lock down this hard.)

Homework reminder: memorize the `ip` subcommands and what they do, switch your display driver to VirtIO GPU, and install the Proxmox client on your VM - from your own VM's console, not from inside an SSH session on another machine (you won't have admin rights there).